

Lecture 5: Unsupervised Learning
PCA, t-SNE, Clustering, Autoencoders
Phase Diagrams of Classical Spin Models without Labels

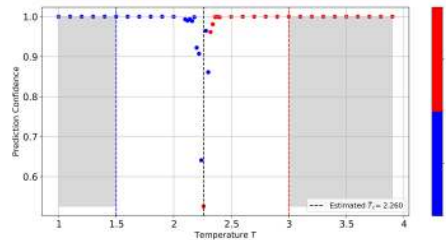
Deep Learning in Classical and Quantum Physics

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

Recap: Finding T_c with Labels

- a CNN trained on spin configurations labelled "ordered" ($T < 1.5$) and "disordered" ($T > 3$)
- its average confidence is lowest at $T = 2.26$ ($L = 50$); exact value $T_c \approx 2.269$
- but the labels came from us: we knew that there are two phases, and where each of them lies



Today: what can we learn from data without labels?

1

Learning without Labels



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Unsupervised Learning

Supervised learning needs labels, and labels are expensive (every sample labelled, often by an expert), contain what we already know (the ten digits, the two phases of a magnet), and in discovery the categories are often unknown, or whether there are any.

Supervised

data: pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$

goal: a function $f(\mathbf{x}) \approx y$, i.e. learn $p(y \mid \mathbf{x})$

we fix the question in advance: which cell type? which phase?

Unsupervised

data: samples only, $\{\mathbf{x}_i\}_{i=1}^n$

goal: learn about $p(\mathbf{x})$, the distribution the data come from

no question fixed in advance: we want to **understand the data**

Understanding the data, in a broad sense:

question about the data	method
which few variables describe a sample?	dimensionality reduction
how many kinds of samples are there?	clustering
what is typical, and what is unusual?	anomaly detection
how are the samples distributed? can we draw new ones?	density estimation, generative models

The method proposes a structure; deciding what it means is ours.

High-Dimensional Data

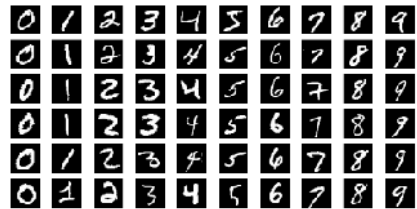
every sample is a vector $\mathbf{x} \in \mathbb{R}^D$: an MNIST digit has $D = 28 \cdot 28 = 784$ pixels, an Ising configuration $D = L^2$ spins

We cannot look at 784 dimensions. But the data do not fill $\mathbb{R}^{784}$: pick 784 random pixel values and you get noise, never a digit. Real images lie in a thin, low-dimensional part of the space.

So the analysis starts with **dimensionality reduction**: project the data on a smaller space,

$$\mathbf{x} \in \mathbb{R}^D \;\mapsto\; \mathbf{z} \in \mathbb{R}^d, \qquad d \ll D,$$

keeping their structure; then look at the points $\mathbf{z}_i$: do they form groups?



2

Principal Component Analysis

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Principal Component Analysis: a Linear Projection

Data $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^D$, with mean $\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$ (for MNIST: the average image).

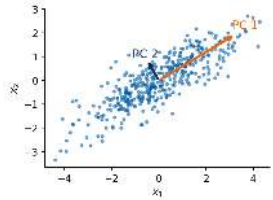
- covariance matrix of the features, $D \times D$:

$$C_{jl} = \frac{1}{n}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)(x_{il} - \bar{x}_l), \qquad j, l = 1, \ldots, D$$

  x_{ij}: feature j (e.g. pixel j) of sample i; C_{jj}: the variance of feature j; C_{jl}: how features j and l vary together over the data

- diagonalise it: $C\,\mathbf{v}_a = \lambda_a \mathbf{v}_a$, $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$; λ_a is the variance of the data along the unit vector $\mathbf{v}_a$
- keep the m eigenvectors with the largest eigenvalues, the **principal components**
- project every sample on them: $z_{ia} = \mathbf{v}_a \cdot (\mathbf{x}_i - \bar{\mathbf{x}})$, $a = 1, \ldots, m$

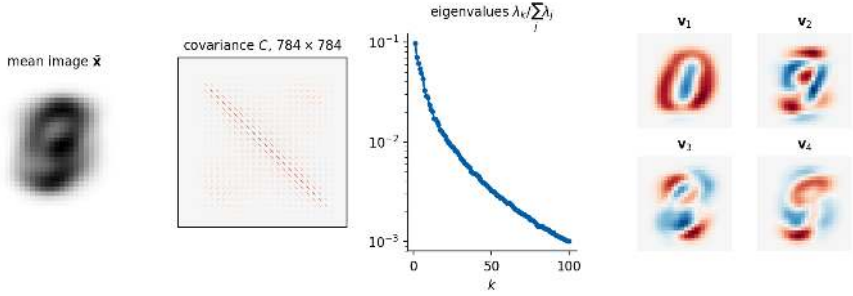
In matrix form $\mathbf{z} = V^\top(\mathbf{x} - \bar{\mathbf{x}})$, with $V = (\mathbf{v}_1, \ldots, \mathbf{v}_m)$ the $D \times m$ matrix of the kept eigenvectors: a linear map. (Features divided by their standard deviations first: C becomes the *correlation* matrix.)



400 correlated points: PC 1 carries 93.4% of the variance

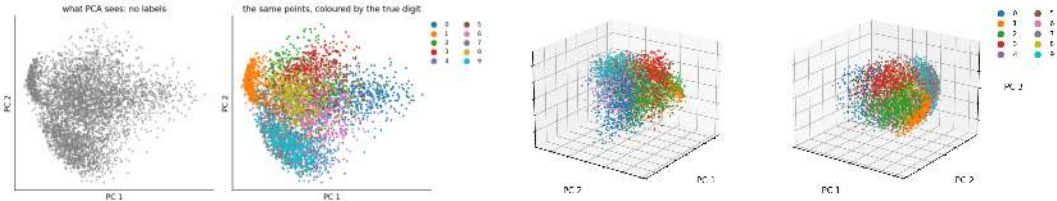
K. Pearson, *On lines and planes of closest fit to systems of points in space*, Phil. Mag. **2**, 559 (1901), doi:10.1080/14786440109462720; H. Hotelling, *Analysis of a complex of statistical variables into principal components*, J. Educ. Psychol. **24**, 417 (1933), doi:10.1037/h0071325

PCA on MNIST: the Ingredients



- 60 000 training images, each a vector of 784 pixels; C is 784×784: how two pixels vary together over the images
- the eigenvalues fall fast: 2 components keep 17% of the variance, 10 keep 49%, 87 components keep 90%
- the eigenvectors are themselves images: patterns of pixels that vary together

PCA on MNIST

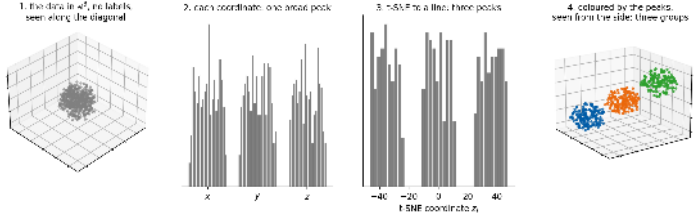


- 5000 test digits projected on the first two components (left): they keep 17% of the variance; three components (right, two viewing angles) keep 23%; 50 components keep 82%
- is there clustering? some digits form separate groups (0, 1), the third direction separates a few more, but most overlap: the digits do not lie in a flat, low-dimensional subspace, and a linear projection is too crude

3
t-SNE

JAGIELLONIAN UNIVERSITY
IN KRAKOW

t-SNE: from 3D to a Line



- 450 points in $\mathbb{R}^3$, given *without* labels; seen along the diagonal they form one blob (1), and along every coordinate the values overlap (2): no single coordinate shows how many groups there are (in 3D one can still rotate the picture; in 100 dimensions there is no view to look from)
- t-SNE gives each point one number z_i, keeping close points close; on the line three separate peaks appear (3)
- colouring the points by their peak reveals three groups in 3D (4), exactly the three that generated the data
- what is kept: who is next to whom; what is not: large distances, so the gaps between the peaks are not the distances between the groups

t-SNE: Similarities in the Original Space

A linear projection to 2D squeezes curved structure together. t-SNE asks instead: **neighbours in $\mathbb{R}^D$ stay neighbours in the 2D map**. The positions in the map are the parameters, found by gradient descent; there is no network.

$\mathbf{x}_i \in \mathbb{R}^D$, $i = 1, \ldots, n$: the original data instances (one MNIST image: $D = 784$). Each pair is reduced to one number, the squared distance

$$\|\mathbf{x}_i - \mathbf{x}_j\|^2 = \sum_{k=1}^{D} (x_{ik} - x_{jk})^2,$$

a sum over the D features (for images i and j: over their pixels k). For instance i, turn its distances to all other instances into weights with a Gaussian of width σ_i, and divide by their sum:

$$p_{j|i} = \frac{e^{-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma_i^2}}{\sum_{l \neq i} e^{-\|\mathbf{x}_i - \mathbf{x}_l\|^2/2\sigma_i^2}}, \qquad p_{i|i} = 0$$

the probability that $\mathbf{x}_i$ would pick $\mathbf{x}_j$ as its neighbour, out of the $n-1$ other instances: a sum over the data set, not an integral over $\mathbb{R}^D$. σ_i is set by the **perplexity**, roughly the number of neighbours each instance keeps (here 30). Symmetrised over the pair:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}, \qquad \sum_{i \neq j} p_{ij} = 1$$

$P = \{p_{ij}\}$

a probability distribution over all pairs of instances: how likely i and j are neighbours *in the original space*

t-SNE: Similarities in the Map

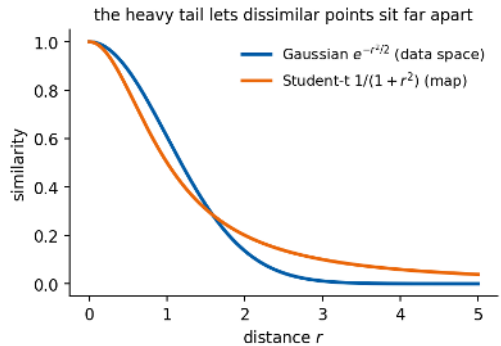
$\mathbf{z}_i \in \mathbb{R}^d$: the point that represents instance i in a **low-dimensional space whose dimension d we choose**; $d = 2$ to plot it. The $\mathbf{z}_i$ are the unknowns.

Similarity of two map points, with a Student-t:

$$q_{ij} = \frac{(1 + \|\mathbf{z}_i - \mathbf{z}_j\|^2)^{-1}}{\sum_{l \neq m} (1 + \|\mathbf{z}_l - \mathbf{z}_m\|^2)^{-1}}$$

with $\|\mathbf{z}_i - \mathbf{z}_j\|^2 = \sum_{a=1}^{d} (z_{ia} - z_{ja})^2$, a sum over the d map coordinates; the denominator sums over all pairs, so $\sum_{i \neq j} q_{ij} = 1$. $Q = \{q_{ij}\}$: the same kind of distribution over pairs, *in the map*.

The heavy tail. In 784 dimensions an instance can have many neighbours at similar distances; in 2D there is no room for all of them (the *crowding problem*). The Student-t decays more slowly, so moderately distant instances sit further apart in the map, and gaps open between the groups.



t-SNE: the Loss

Two distributions over pairs of instances: P from the original space, Q from the map. Move the map points $\mathbf{z}_i$ by gradient descent until Q matches P, i.e. minimise their Kullback–Leibler divergence:

$$\mathcal{L}(\mathbf{z}_1, \ldots, \mathbf{z}_n) = \mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

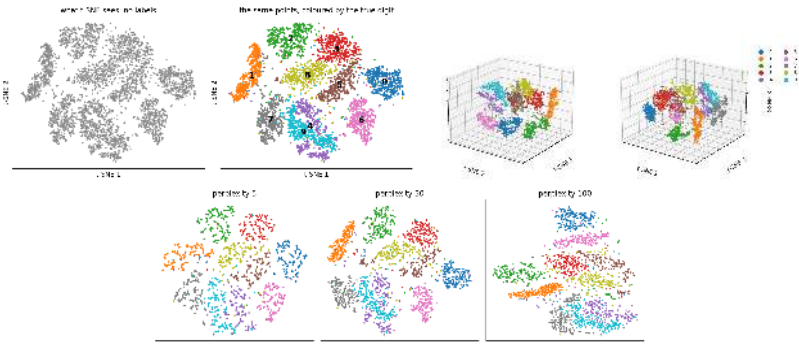
- neighbours placed far apart (p_{ij} large, q_{ij} small): large cost
- distant instances placed close (p_{ij} small): small cost
- so t-SNE keeps the **local** structure; distances between groups in the map carry little meaning

The result is a set of 2D points, one per instance, that we can plot and inspect.

L. van der Maaten, G. Hinton, *Visualizing Data using t-SNE*, J. Mach. Learn. Res. **9**, 2579 (2008),

jmlr.org/papers/v9/vandermaaten08a

t-SNE on MNIST



top: the 5000 digits in 2D (labels only colour the right panel) and in 3D; bottom: 2000 digits at three perplexities

- ten groups appear without any label; small perplexity keeps only very close neighbours, large perplexity smooths the map
- **meaningless:** the position of a cluster (a rotated, mirrored or shifted map has the same loss; another seed moves it); the **separation** between clusters (distant pairs have $p_{ij} \approx 0$); cluster **sizes** (σ_i adapts to the density); larger distances **inside** a cluster
- **reliable:** which instances are close neighbours

4

Clustering



JAGIELLONIAN UNIVERSITY
IN KRAKOW

k-means Clustering

Goal: split the n data instances into k groups (clusters), such that instances in the same group are close to each other.

Each cluster S_c (the set of its instances) has a centre $\boldsymbol{\mu}_c$; k-means minimises the total squared distance of the instances to their centres:

$$\mathcal{L} = \sum_{c=1}^{k} \sum_{i \in S_c} \|\mathbf{x}_i - \boldsymbol{\mu}_c\|^2$$

The algorithm

start from k random centres, then alternate:

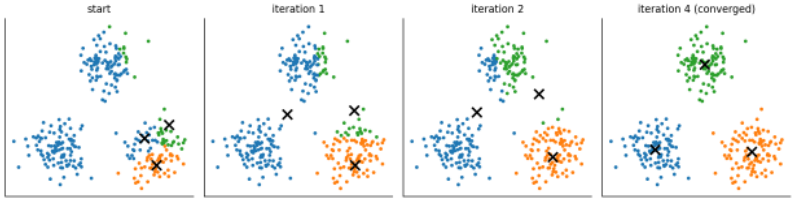
- *assign:* each instance joins its nearest centre
- *update:* each centre moves to the mean of its instances

until no instance changes cluster

- each move can only lower $\mathcal{L}$, so the algorithm always stops
- it can stop in a poor solution: run it from several random starts and keep the lowest $\mathcal{L}$
- k is chosen by us

S. P. Lloyd, *Least squares quantization in PCM*, IEEE Trans. Inf. Theory **28**, 129 (1982), doi:10.1109/TIT.1982.1056489

k-means, Iteration by Iteration



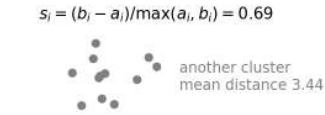
The iterative algorithm:

1. choose k and place the k centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ at random (e.g. at k randomly chosen instances)
2. **assign:** each instance joins the cluster of its nearest centre, $S_c = \{\, i : \|\mathbf{x}_i - \boldsymbol{\mu}_c\| \leq \|\mathbf{x}_i - \boldsymbol{\mu}_{c'}\| \text{ for all } c' \,\}$
3. **update:** each centre moves to the mean of its instances, $\boldsymbol{\mu}_c = \frac{1}{|S_c|} \sum_{i \in S_c} \mathbf{x}_i$
4. repeat 2 and 3 until no instance changes cluster

Above: three groups of points in 2D, the three centres (crosses) placed badly on purpose; colours show the assignment. The loss never increases, and after a few iterations nothing changes.

The Silhouette Score

In a good clustering each instance is **close to its own cluster** and **far from the other clusters**. The silhouette measures this for every instance, by comparing two mean distances.



For an instance $\mathbf{x}_i$ in cluster S_c:

- a_i: mean distance from $\mathbf{x}_i$ to the other instances of S_c
- b_i: mean distance from $\mathbf{x}_i$ to the instances of the nearest other cluster

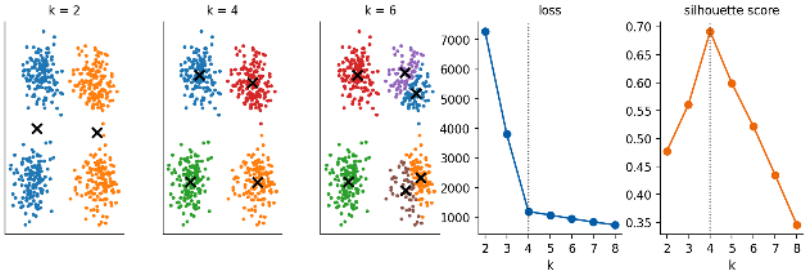
$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$$

- $s_i \approx 1$: well inside its cluster, far from the others
- $s_i \approx 0$: on the border between two clusters
- $s_i < 0$: closer to another cluster, probably assigned wrongly

$\bar{s}(k) = \frac{1}{n}\sum_i s_i$ for the clustering with k clusters; choose the k with the largest $\bar{s}(k)$.

P. J. Rousseeuw, *Silhouettes: A graphical aid to the interpretation and validation of cluster analysis*, J. Comput. Appl. Math. **20**, 53 (1987), doi:10.1016/0377-0427(87)90125-7

Choosing the Number of Clusters

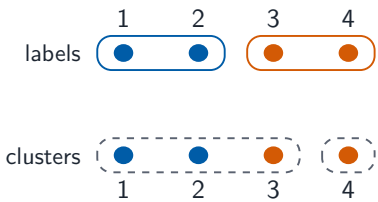


k-means needs k from us. For every $k = 2, 3, \ldots, 8$: run k-means (10 random starts, keep the lowest loss), record the loss $\mathcal{L}(k)$ and the mean silhouette score $\bar{s}(k)$, and plot both against k. Four groups of points, so the right answer is known:

- **elbow:** $\mathcal{L}(k)$ always decreases with k (more centres, shorter distances); it decreases fast up to the right k and slowly after it: the bend is at $k = 4$
- **silhouette:** $\bar{s}(k)$ is highest at $k = 4$
- too small k merges groups, too large k splits them (left panels)

Rand Index

How well does a clustering recover known classes? Compare two partitions of the same n instances, the true labels and the clusters, *pair by pair*; the names of the groups do not matter.

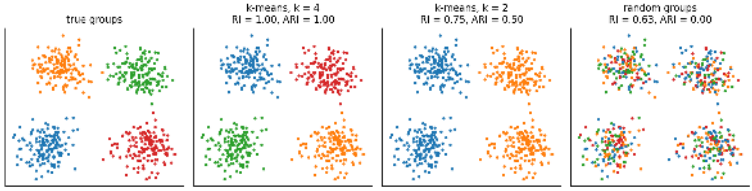


pair	labels	clusters	agree
1–2	same	same	✓
1–3	different	same	×
1–4	different	different	✓
2–3	different	same	×
2–4	different	different	✓
3–4	same	different	×

A pair **agrees** if both partitions put it in the same group, or both in different groups. The **Rand index** is the fraction of agreeing pairs, out of all $n(n-1)/2$:

$$\mathrm{RI} = \frac{\text{agreeing pairs}}{n(n-1)/2} = \frac{3}{6} \quad \text{here;} \qquad \mathrm{RI} = 1 \text{ for identical partitions}$$

Adjusted Rand Index



- random groups already reach a high RI (0.63 here): most pairs are in different groups in both partitions
- the adjusted Rand index corrects for chance:

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{1 - \mathbb{E}[\mathrm{RI}]},$$

  with $\mathbb{E}[\mathrm{RI}]$ the mean RI when the instances are reassigned at random, keeping the group sizes
- **ARI = 1**: the same partition; **ARI ≈ 0**: no better than chance; it can be negative

L. Hubert and P. Arabie, *Comparing partitions*, J. Classification **2**, 193 (1985), doi:10.1007/BF01908075

k-means on MNIST

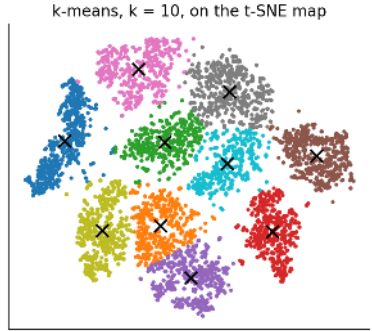
Split the samples into k groups $S_1, \ldots, S_k$ with centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$, minimising

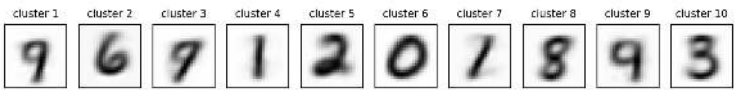
$$\mathcal{L} = \sum_{c=1}^{k} \sum_{i \in S_c} \|\mathbf{x}_i - \boldsymbol{\mu}_c\|^2$$

Repeat until nothing changes:

- assign every sample to its nearest centre
- move every centre to the mean of its samples

$k = 10$, the number of digits. Agreement with the true digits, adjusted Rand index: 0.38 on 50 PCA components, 0.76 on the t-SNE map.

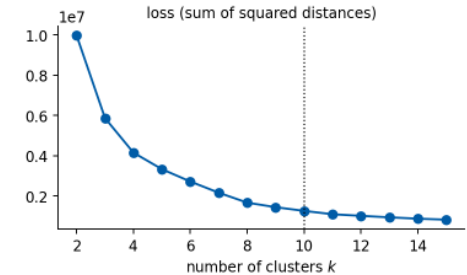


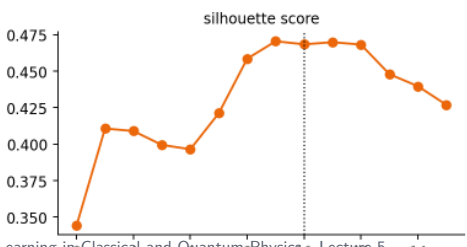


the ten centres found on 50 PCA components, shown as images

S. P. Lloyd, *Least squares quantization in PCM*, IEEE Trans. Inf. Theory **28**, 129 (1982), doi:10.1109/TIT.1982.1056489

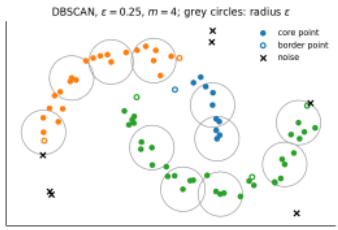
Choosing k on MNIST





- the same two guides on the t-SNE map of the 5000 digits: the loss $\mathcal{L}(k)$ and the mean silhouette $\bar{s}(k) = \frac{1}{n}\sum_i \frac{b_i - a_i}{\max(a_i, b_i)}$ (a_i: mean distance to its own cluster, b_i: to the nearest other cluster)
- dotted line: $k = 10$, the number of digits; drawn only for comparison, the guides do not use the labels
- no sharp elbow; the silhouette is highest at $k = 9$, and almost equal at $k = 10$ and 11: a flat maximum next to the 10 digits (at $k = 10$ the 4s and 9s are mixed across two clusters)
- on real data the guides suggest a range of

DBSCAN: Density-Based Spatial Clustering of Applications with Noise



k-means cuts the space into convex cells, so it finds round clusters: on two half-moons k-means ($k = 2$) gives ARI 0.24, DBSCAN 1.00

Two parameters: a radius ε and a minimum number of points m.

- **core point:** at least m points within distance ε (grey circles)
- **border point:** not core, but within ε of a core point
- **noise:** all other points (crosses)

The algorithm:

1. count the neighbours within ε of every point (itself included) and mark the core points
2. take a core point not yet in a cluster and start a new cluster with it
3. add every point within ε of a core point of the cluster; repeat until no point is added: the cluster grows through the dense region and stops at its border points
4. repeat 2 and 3 until every core point is in a cluster; the points never reached are noise

No number of clusters is needed, and clusters can have any shape.

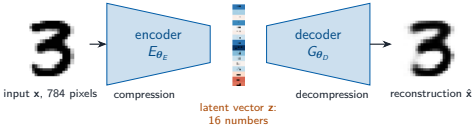
M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*, KDD 1996, AAAI proceedings

5
Autoencoders

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Autoencoder

Motivation. PCA compresses with a straight (linear) projection: MNIST needs 87 components for 90% of the variance. Can a network learn a better, non-linear compression? *If a network can rebuild an image from only d numbers, those numbers must contain what is essential about it.*



- **encoder** E_{θ_E}: a neural network with D inputs (the size of the data, 784 pixels) and d outputs (the latent-space size, chosen by us, here 16); its own parameters (weights and biases) $\boldsymbol{\theta}_E$; $\mathbf{z} = E_{\theta_E}(\mathbf{x}) \in \mathbb{R}^d$
- **decoder** G_{θ_D}: a second neural network with d inputs and D outputs, with its own parameters $\boldsymbol{\theta}_D$; $\hat{\mathbf{x}} = G_{\theta_D}(\mathbf{z}) \in \mathbb{R}^D$
- both are trained together on the images alone, with no labels: every input is its own target
- the bottleneck $d \ll 784$ forces the encoder to extract the crucial features of the data, those the decoder needs to rebuild the image; they are encoded in $\mathbf{z}$, a learned, low-dimensional **latent-space representation**
- a linear autoencoder finds the same subspace as PCA; non-linear layers can follow curved structure

G. E. Hinton, R. R. Salakhutdinov, *Reducing the Dimensionality of Data with Neural Networks*, Science **313**, 504 (2006), doi:10.1126/science.1127647

The Autoencoder Loss



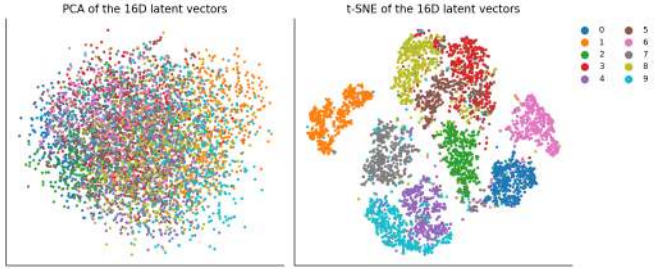
A **loss function** is one number that measures how badly the network does on the training data, for given values of its parameters; training means finding the parameters that make it smallest. Here it compares each input with its reconstruction $\hat{\mathbf{x}}_i = G_{\theta_D}(E_{\theta_E}(\mathbf{x}_i))$ **pixel by pixel**:

$$\mathcal{L}(\boldsymbol{\theta}_E, \boldsymbol{\theta}_D) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{D}\sum_{j=1}^{D}\left(x_{ij} - \hat{x}_{ij}\right)^2$$

i: image, j: pixel. The right panel is the squared difference for one digit; $\mathcal{L}$ is its average over all pixels and all images.

- $\mathcal{L} = 0$ only if every pixel is rebuilt exactly
- no labels: the target is the input itself
- $\boldsymbol{\theta}_E$ and $\boldsymbol{\theta}_D$ are updated together by gradient descent and backpropagation, as for any network; for MNIST: dense layers $784 \to 256 \to 64 \to d$ and back

Compression and the Latent Space



5000 test digits, 16 latent numbers each, drawn in 2D with PCA and with t-SNE; colours: true digits, used only for plotting

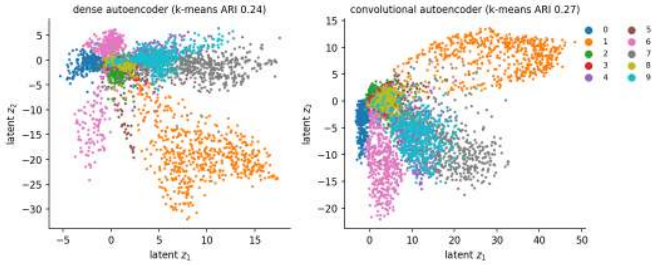
- **compression:** a 784-pixel image becomes $d = 16$ numbers $\mathbf{z} = E_{\theta_E}(\mathbf{x})$ (49 times fewer), from which the decoder rebuilds it
- the loss does not ask for it, but similar images end up with nearby $\mathbf{z}$: the decoder must rebuild them alike

k-means ($k = 10$) against the true digits, ARI:

	directly	t-SNE map
pixels (50 PCA comp.)	0.38	0.76
16 latent numbers	0.28	0.70

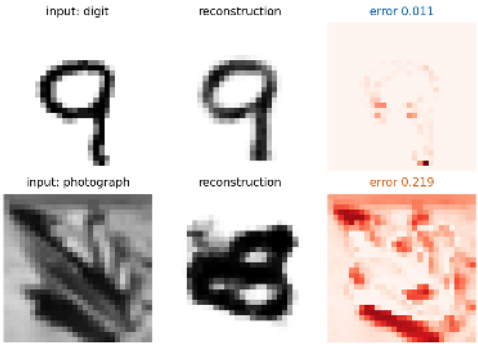
The digits are kept in the latent space, but not as the round groups that k-means looks for, t-SNE

Autoencoder on MNIST: Dense and Convolutional



- autoencoders with only $d = 2$ latent numbers, so the latent space can be drawn directly, without PCA or t-SNE; colours: true digits
- **dense** (left): sees an image as a flat list of 784 pixels; **convolutional** (right): two convolutions with stride 2 ($28 \to 14 \to 7$ pixels, 16 and 32 channels), as in a CNN
- k-means with $k = 10$ in the 2D latent space: adjusted Rand index 0.24 (dense), 0.27 (convolutional)
- with the same two numbers, the convolutional encoder separates the digits better: it compares neighbouring pixels, as a CNN does

Anomaly Detection with an Autoencoder

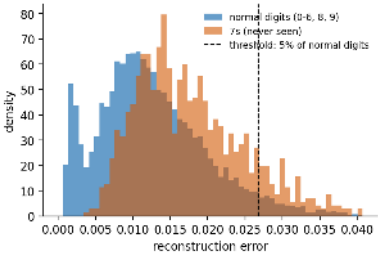


autoencoder trained on all MNIST digits; a typical digit and a typical photograph (median errors)

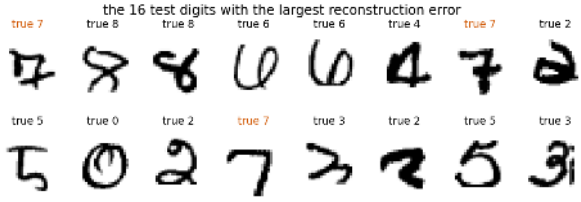
The task: find samples that are unlike the rest, when we have no examples of them.

- a well-trained autoencoder has a **small loss** on data like its training data: it rebuilds them well
- very odd data are rebuilt badly: the reconstruction error $\|\mathbf{x} - G_{\theta_D}(E_{\theta_E}(\mathbf{x}))\|^2$ is **large**
- a large error means the sample is *surprising* for the model: an outlier, an **anomaly**
- **decision rule:** choose a threshold on the error, e.g. so that only 5% of normal samples exceed it (5% false alarms), and flag every sample above it

A Subtle Anomaly: the Digit 7

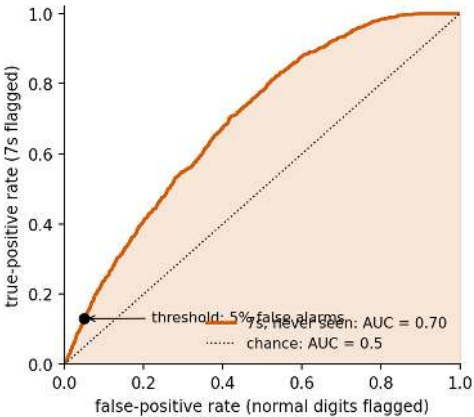


- **experiment:** train the autoencoder on all digits *except the 7s*, then compute the error of every test digit
- **histogram:** horizontal axis = reconstruction error; blue: the digits it was trained on, orange: the 7s, never seen
- the two populations overlap: the threshold (dashed, 5% false alarms) flags only 13% of the 7s; AUC 0.70
- a 7 shares strokes with 1 and 9, so many 7s are rebuilt well



the 16 test digits with the largest error: some are 7s (orange labels), others are badly written digits of the trained classes

ROC Curve and AUC



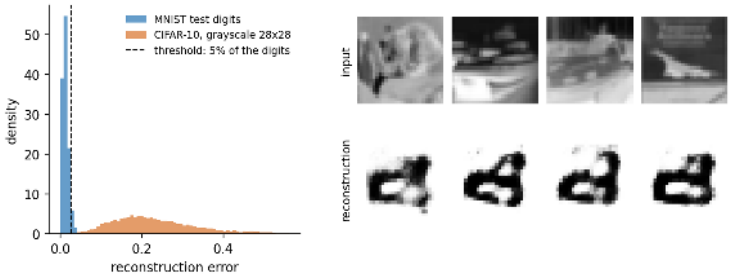
Each threshold on the anomaly score gives one decision rule. For each:

- **true-positive rate** (TPR): fraction of anomalies above the threshold, i.e. found
- **false-positive rate** (FPR): fraction of normal samples above it, i.e. false alarms

Lowering the threshold traces the **ROC curve** (receiver operating characteristic) from $(0, 0)$ to $(1, 1)$. Chance lies on the diagonal; a perfect score goes straight up to TPR $= 1$.

AUC: the area under the curve, 0.5 for chance, 1 for a perfect separation; equal to the probability that a random anomaly has a larger score than a random normal sample.

A Clear Anomaly: Photographs



- **experiment:** the autoencoder trained on all MNIST digits receives CIFAR-10 photographs (aeroplanes, cars, animals, ships, trucks), converted to 28×28 grayscale
- the two populations barely overlap: the threshold that flags 5% of the digits flags 99.8% of the photographs; AUC 0.9997
- the reconstructions (right) look like blurred digits: the decoder can only produce what it learned

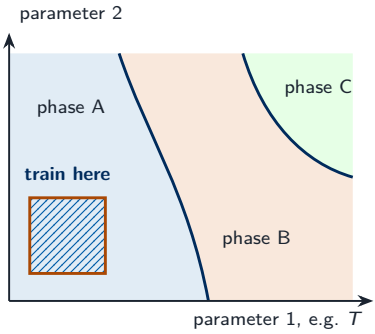
An anomaly far from the training data is easy to detect; one that resembles it is hard.

6

Anomaly Detection for Studies of Phase Diagrams

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Anomaly Detection on a Phase Diagram



schematic phase diagram; thick lines: phase boundaries

- **train** the autoencoder only on samples from a small, well-understood region inside one phase (hatched)
- **scan:** compute the reconstruction error of samples everywhere in the diagram
- **plot** the error as a map: low inside the trained phase, high elsewhere
- **phase boundaries** appear where the error changes fastest, without labels and without an order parameter

Kottmann et al. mapped the phase diagram of the one-dimensional extended Bose–Hubbard model in this way.

K. Kottmann, P. Huembeli, M. Lewenstein, A. Acín, *Unsupervised Phase Discovery with Deep Anomaly Detection*, Phys. Rev. Lett. **125**, 170603 (2020), doi:10.1103/PhysRevLett.125.170603

Model by Model

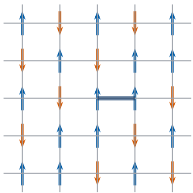
The same protocol for every model: **physics first** (Hamiltonian, phases, the observables physicists constructed), then the **unsupervised methods** (PCA, autoencoder anomaly detection), then a comparison with what is known.

	model	what makes it interesting
1	2D Ising	one phase transition, exactly known; the reference
2	J_1–J_2 Ising	competing couplings; the magnetisation misses a whole phase

Model 1

The 2D Ising Model

The 2D Ising Model, Once More

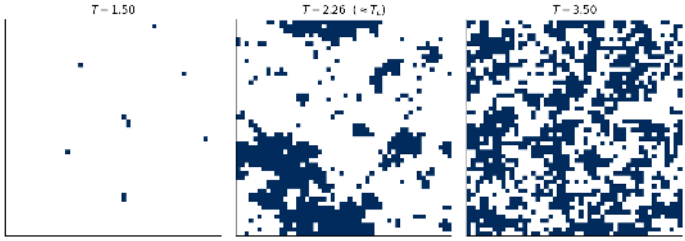


bond $\langle i,j \rangle$

spins $s_i = \pm 1$ on an $L \times L$ lattice, periodic boundaries, $N = L^2$

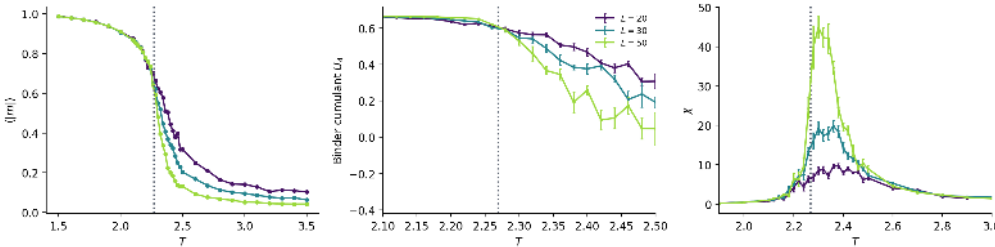
$$H(\mathbf{s}) = -J \sum_{\langle i,j \rangle} s_i s_j, \qquad p(\mathbf{s}) = \frac{e^{-\beta H(\mathbf{s})}}{Z}, \qquad Z = \sum_{\mathbf{s}} e^{-\beta H(\mathbf{s})}$$

thermal averages: $\langle O \rangle = \sum_{\mathbf{s}} O(\mathbf{s})\, p(\mathbf{s})$, sampled by Monte Carlo; exact $T_c = 2/\ln(1+\sqrt{2}) = 2.269$ (Onsager)



Lecture 4: a CNN with labels found T_c. Today: the same data, no labels.

The Reference: T_c from Finite-Size Scaling



$$m = \frac{1}{N}\sum_i s_i, \quad U_4 = 1 - \frac{\langle m^4 \rangle}{3\langle m^2 \rangle^2}, \quad \chi = \frac{N}{T}\left(\langle m^2 \rangle - \langle |m| \rangle^2\right)$$

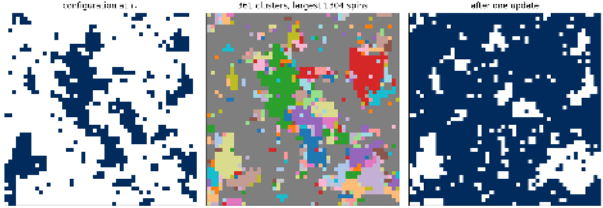
- U_4 curves for different L **cross at** T_c: 2.250 ($L = 20/30$), 2.278 ($L = 30/50$)
- the peak of χ moves as $T_c(L) = T_c + a/L$: 2.371, 2.362, 2.303; fit: $T_c = 2.269$, exact 2.2692

K. Binder, Z. Phys. B **43**, 119 (1981), doi:10.1007/BF01293604; M. E. J. Newman, G. T. Barkema, *Monte Carlo Methods in Statistical Physics*, Oxford University Press (1999)

Sampling near T_c: Flip Whole Clusters

Near T_c single-spin Metropolis flips are rejected inside large domains: successive samples are almost copies (*critical slowing down*). **Swendsen–Wang update:**

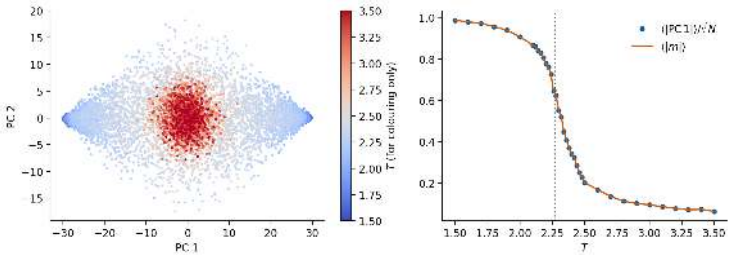
1. bond each pair of parallel neighbours with probability $p = 1 - e^{-2\beta J}$
2. find the clusters connected by bonds (middle panel)
3. flip every cluster with probability $1/2$ (right panel)



exact: $e^{\beta J s_i s_j} = e^{\beta J}\big[(1-p) + p\,\delta_{s_i s_j}\big]$; $L = 50$ at T_c: autocorrelation time 26.4 Metropolis sweeps vs 3.4 cluster updates R. H. Swendsen, J.-S. Wang, Phys. Rev. Lett. **58**, 86 (1987), doi:10.1103/PhysRevLett.58.86

PCA Finds the Order Parameter

9472 configurations (30×30, 37 temperatures), each a vector $\mathbf{s} \in \{-1, +1\}^{900}$; PCA receives only the spins, no temperature, no phase.



$z_k = \mathbf{v}_k \cdot (\mathbf{s} - \bar{\mathbf{s}})$; right: $\langle |z_1| \rangle / \sqrt{N}$

- low T: two groups at $z_1 \approx \pm 30 = \pm\sqrt{N}$, all spins up or all down; high T: one group around 0
- $\langle |z_1| \rangle / \sqrt{N}$ lies on the magnetisation $\langle |m| \rangle$; PC 1 carries 37% of the variance

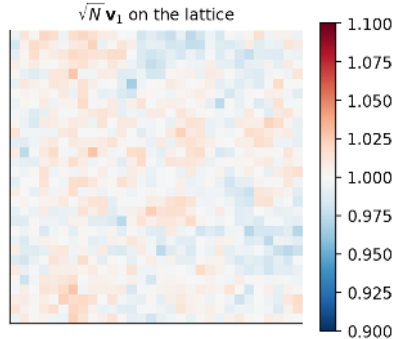
L. Wang, Phys. Rev. B **94**, 195105 (2016), doi:10.1103/PhysRevB.94.195105; S. J. Wetzel, Phys. Rev. E **96**, 022140 (2017), doi:10.1103/PhysRevE.96.022140

The First Principal Component Is the Magnetisation

- below T_c all pairs of spins are correlated, $\langle s_i s_j \rangle \approx M^2$: the covariance matrix has a large uniform block, and its leading eigenvector is $\mathbf{v}_1 \approx (1, \ldots, 1)/\sqrt{N}$
- projecting on it:

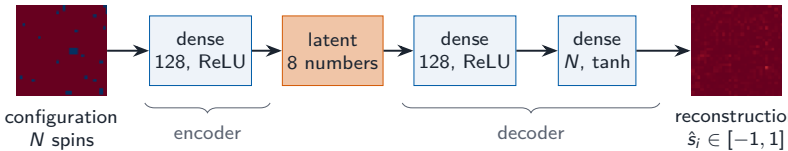
$$z_1 = \mathbf{v}_1 \cdot (\mathbf{s} - \bar{\mathbf{s}}) \approx \frac{1}{\sqrt{N}} \sum_{i=1}^{N} s_i = \sqrt{N}\, m$$

- measured: the entries of $\sqrt{N}\,\mathbf{v}_1$ are 1 with a spread of 1%; the correlation of $|z_1|$ with $|m|$ is 0.99996
- the Binder crossings computed from z_1 agree with those from m



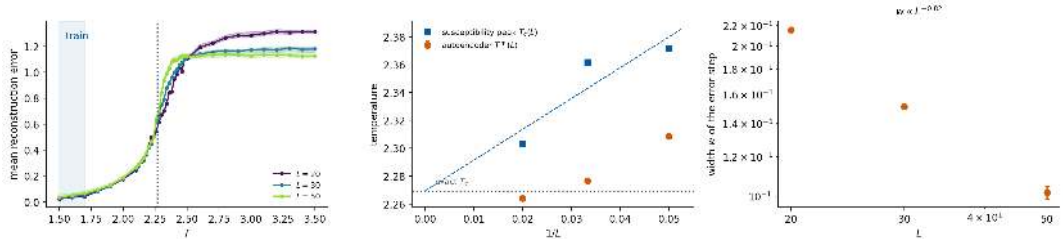
The Ising order parameter is linear in the spins, and a linear method finds linear structure.

Autoencoder on Spin Configurations: the Setup



- **input:** one configuration $\mathbf{s} \in \{-1, +1\}^N$, $N = L^2$, for $L = 20, 30, 50$; the same architecture for every L
- **training data:** only the 768 configurations with $T \leq 1.7$, deep in the ordered phase (Yang: $M(1.7) = 0.970$); no temperatures and no labels are given to the network
- **loss:** $\frac{1}{N}\|\mathbf{s} - \hat{\mathbf{s}}\|^2$, Adam, 200 epochs, three random seeds per L
- **measurement:** the mean reconstruction error at each of the 37 temperatures

The Autoencoder Signal Sharpens with the System Size

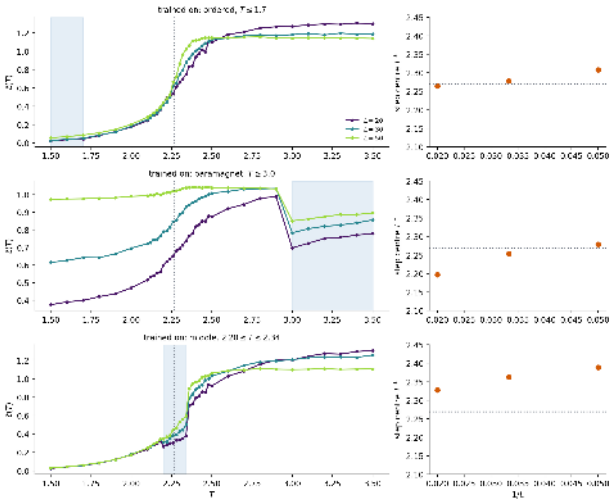


reconstruction error of one configuration $\varepsilon(\mathbf{s}) = \frac{1}{N}\sum_i (s_i - \hat{s}_i)^2$; $\bar{\varepsilon}(T) = \langle \varepsilon \rangle_T$, its average over the configurations at T; fit $\bar{\varepsilon}(T) = a + b\tanh\big((T - T^*)/w\big)$: centre T^*, width w

L	T^*	$T^* - T_c$	w
20	2.308	$+0.039$	0.215
30	2.277	$+0.008$	0.151
50	2.264	-0.005	0.102

- $T^* \to T_c$ as L grows; three seeds agree to ± 0.002
- $w \propto L^{-0.82}$; the finite-size rounding of the transition scales as $L^{-1/\nu} = L^{-1}$ (three sizes only: indicative)
- the autoencoder is not a thermodynamic observable: no derived finite-size law, so we do not extrapolate T^*

The Training Region Decides the Signal

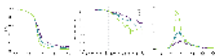
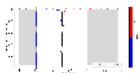
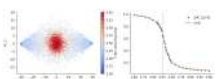


rows: trained on $T \leq 1.7$, $T \geq 3.0$, $2.20 \leq T \leq 2.34$ (shaded); left $\bar{\varepsilon}(T)$, right T^* vs $1/L$

- **ordered:** $T^* = 2.308, 2.278, 2.264 \to T_c = 2.269$
- **paramagnet:** trained on noise, no structure is learned; for $L = 50$ the error is flat
- **middle:** the jump sits at the edge of the training window

Train on a well-structured, well-understood phase.

Four Ways to Find T_c

	method	what we gave it	what it found
	finite-size scaling	the order parameter m	$T_c = 2.269$ (exact 2.2692)
	CNN (Lecture 4)	labels far from T_c	lowest confidence at 2.26 ($L = 50$)
	PCA	spins only	the magnetisation
	autoencoder	examples of one phase	$T^* = 2.264$ at $L = 50$, sharpening with L

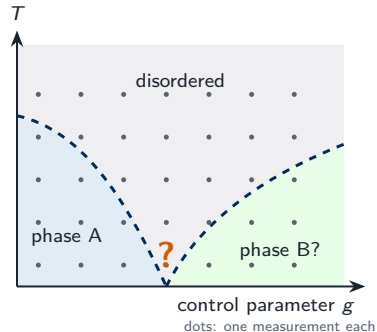
The less we tell the method, the more of the discovery is its own; explaining the result is still ours.

Model 2

A Frustrated Magnet: the J_1–J_2 Ising Model

Motivation for a Second Model

- for the Ising model we *knew* the answer: one ordered phase, one order parameter m, T_c from Onsager
- in real materials the spins feel **competing interactions**; which order appears, and where, is not known in advance
- a phase diagram is mapped by measuring on a grid of control parameters:
  - **which quantity to plot?** the wrong observable hides a phase
  - **a dense grid is expensive**, in the lab and on the computer
- **classical model:** $s_i = \pm 1$, each configuration a snapshot of the

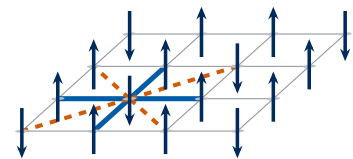


the boundaries (dashed) are unknown; an observable sensitive only to phase A would show phase B as "disordered"; every grid point costs a measurement

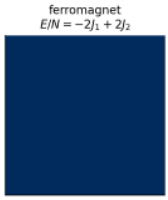
The J_1–J_2 Model: Two Couplings that Compete

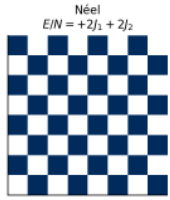
$$H = -J_1 \sum_{\langle i,j \rangle} s_i s_j \; + \; J_2 \sum_{\langle\langle i,j \rangle\rangle} s_i s_j, \qquad g = \frac{J_2}{J_1}$$

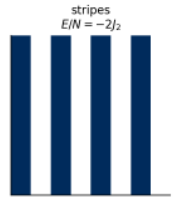
- $J_1 > 0$, 4 nearest neighbours: **ferromagnetic**
- $J_2 > 0$, 4 diagonal neighbours: **antiferromagnetic**
- control parameters g and T: a two-dimensional phase diagram
- exact limits: $g = 0$ is the Ising model, $T_c = 2.269\, J_1$ (Onsager); for $g \to \infty$ the diagonal bonds split the lattice into two independent Ising antiferromagnets, $T_c \to 2.269\, J_2$

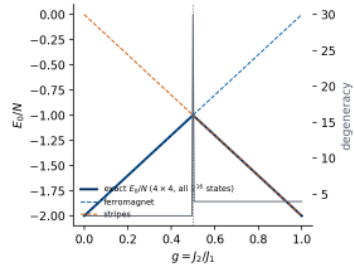


Ground States, Frustration and Mean Field









$T = 0$ (bonds per spin: 2 nearest, 2 diagonal): ferromagnet $E/N = -2J_1 + 2J_2$, stripes $E/N = -2J_2$, Néel $E/N = 2J_1 + 2J_2 \Rightarrow$ ferromagnet for $g < 1/2$, stripes for $g > 1/2$. At $g = 1/2$ both couplings cannot be satisfied: **frustration**; the 4×4 lattice has 30 ground states (2 ferromagnet, 4 stripes).

Mean field (each spin feels the average of its neighbours): $T_{\text{ferro}}^{\text{MF}} = 4(J_1 - J_2)$, $T_{\text{stripes}}^{\text{MF}} = 4J_2$, meeting at $g = 1/2$, $T = 2$. Wrong: fluctuations between the degenerate states destroy the order, and the true $T_c \to 0$ at $g = 1/2$; mean field also gives 4 instead of 2.269 at $g = 0$.

Order Parameters, and What Is Known

Fourier component of the spins at wave vector $\mathbf{k}$: $m_{\mathbf{k}} = \frac{1}{N}\sum_{\mathbf{r}} s_{\mathbf{r}}\, e^{i\mathbf{k}\cdot\mathbf{r}}$

phase	$\mathbf{k}$	order parameter
ferromagnet	$(0,0)$	$m = \frac{1}{N}\sum_{\mathbf{r}} s_{\mathbf{r}}$
stripes	$(\pi,0), (0,\pi)$	$m_{\mathrm{str}} = \sqrt{m_{(\pi,0)}^2 + m_{(0,\pi)}^2}$
Néel	(π,π)	$m_{(\pi,\pi)}$

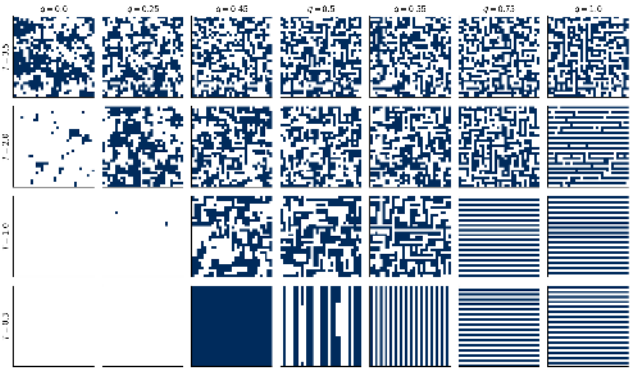
in the stripe phase $m = 0$ **although the system is perfectly ordered**: a scan of m sees it as a paramagnet

- $g < 1/2$: continuous transition to the ferromagnet, 2D Ising universality class, $T_c(g) \to 0$ as $g \to 1/2$
- $g > 1/2$: stripes; weakly first order just above $g = 1/2$, continuous with exponents that change along the line (Ashkin–Teller) above a point g^*, placed at $g^* \approx 0.67$ by Monte Carlo, lower by tensor networks
- near g^* continuous transitions look first order on lattices of practical size

Decades of work, and the right order parameters, were needed.

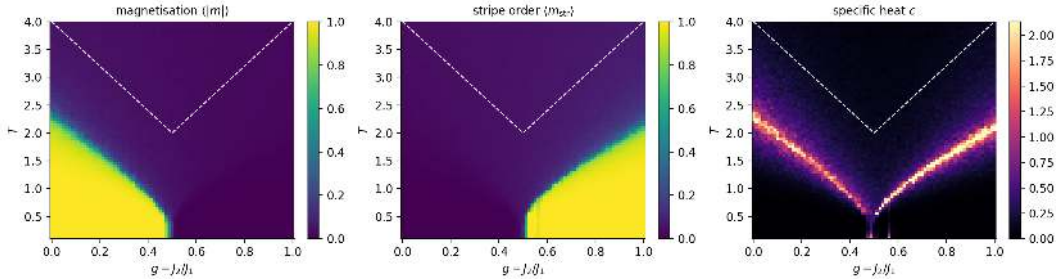
D. P. Landau, K. Binder, Phys. Rev. B **31**, 5946 (1985), doi:10.1103/PhysRevB.31.5946; A. Kalz, A. Honecker, S. Fuchs, T. Pruschke, Eur. Phys. J. B **65**, 533 (2008), arXiv:0805.0983; A. Kalz, A. Honecker, M. Moliner, Phys. Rev. B **84**, 174407 (2011), doi:10.1103/PhysRevB.84.174407; S. Jin, A. Sen, A. W. Sandvik, Phys. Rev. Lett. **108**, 045702 (2012), arXiv:1110.5874; S. Jin, A. Sen, W. Guo, A. W. Sandvik, Phys. Rev. B **87**, 144406 (2013), arXiv:1212.5339; H. Li, L.-P. Yang, Phys. Rev. E **104**, 024118 (2021), arXiv:2103.09464

Snapshots across the (g, T) Plane



- Metropolis Monte Carlo in JAX: all couplings and chains in one compiled program, each chain cooled from $T = 4$ to $T = 0.1$
- 32×32 lattice; 81 values of g, 79 temperatures, 100 snapshots per point
- high T: noise; low T: uniform ($g < 1/2$) or stripes ($g > 1/2$)
- near $g = 1/2$: ferromagnetic and stripe patches coexist; at $g = 1/2$, $T = 0.3$ the stripes contain doubled rows, one of the many degenerate ground-state patterns

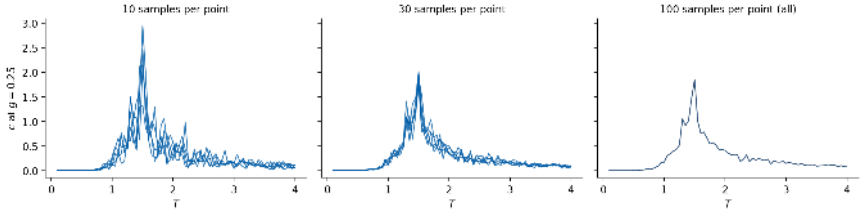
The Observable Decides What We See



$\langle O \rangle = \frac{1}{Z} \sum_{\mathbf{s}} O(\mathbf{s})\, e^{-\beta H(\mathbf{s})}, \quad m = \frac{1}{N} \sum_i s_i,$
$c = \frac{\langle H^2 \rangle - \langle H \rangle^2}{N T^2}$

- $\langle |m| \rangle$: one ordered region, $g < 1/2$; the stripe phase looks like a paramagnet
- $\langle m_{\mathrm{str}} \rangle$: the second ordered region, visible only because we knew which pattern to measure
- c: peaks at every transition, whatever the order, but it is a variance and needs many samples per point

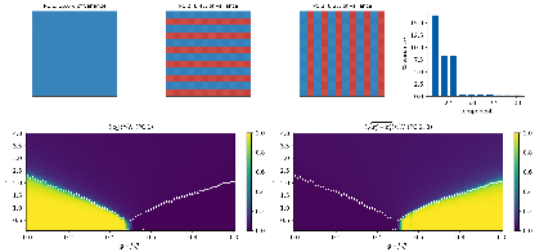
A Dense Scan Is Expensive



- the specific heat at $g = 0.25$ from 10, 30 and all 100 snapshots per temperature; with 10 snapshots the position of the peak jumps between repetitions
- our map: $81 \times 79 = 6399$ grid points $\times$ 100 snapshots $= 639\,900$ configurations; near a transition consecutive Monte Carlo samples are correlated (critical slowing down), so each costs more
- in a laboratory every grid point is a separate preparation of the sample

Wanted: a method that needs no order parameter, works from snapshots, and tells us where a dense scan is worth paying for.

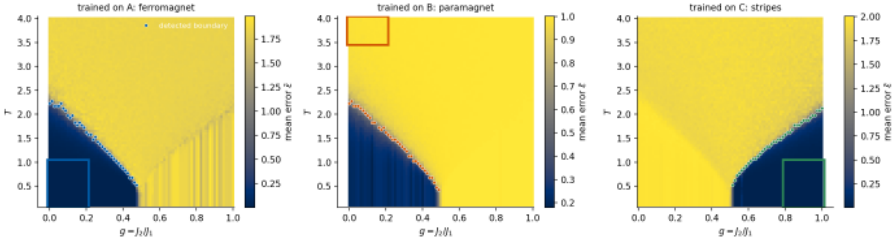
PCA Finds Both Order Parameters



top: $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$; bottom: $\langle |z_1| \rangle / \sqrt{N}$, $\langle \sqrt{z_2^2 + z_3^2} \rangle / \sqrt{N}$, $z_k = \mathbf{v}_k \cdot (\mathbf{s} - \bar{\mathbf{s}})$

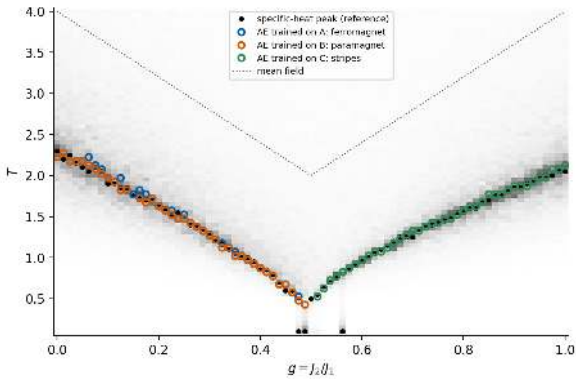
- $\mathbf{v}_1$: uniform ($z_1/\sqrt{N} = m$); $\mathbf{v}_2, \mathbf{v}_3$: the two stripe patterns ($\sqrt{z_2^2 + z_3^2}/\sqrt{N} = m_{\mathrm{str}}$); 16.6%, 8.4%, 8.2% of the variance, all others $< 0.3\%$
- the maps reproduce $\langle |m| \rangle$ and $\langle m_{\mathrm{str}} \rangle$ (correlations 1.000, 0.9999): PCA found both order parameters, including the one the magnetisation missed
- both are linear in the spins, $m_{\mathbf{k}} = \frac{1}{N} \sum_{\mathbf{r}} s_{\mathbf{r}} e^{i\mathbf{k}\cdot\mathbf{r}}$; order that is not linear in the spins is out of reach of PCA

Reconstruction-Error Maps for Three Training Regimes



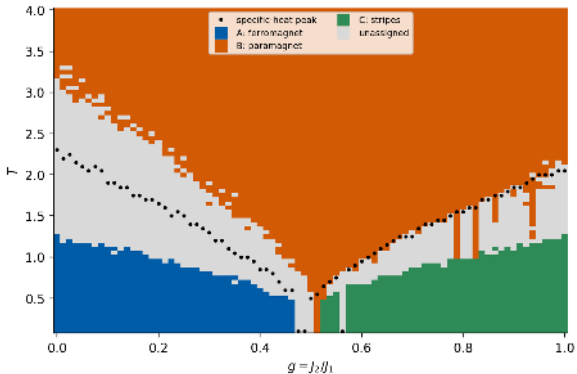
- convolutional autoencoder (4 latent numbers), trained on one corner only (rectangle), with every snapshot also flipped, $\mathbf{s} \to -\mathbf{s}$: the symmetry of H
- **A, ferromagnet** ($g \leq 0.2,\ T \leq 1$): the ferromagnetic phase is rebuilt almost exactly; stripes and paramagnet are both "not ferromagnet"
- **B, paramagnet** ($g \leq 0.2,\ T \geq 3.5$): it learns little more than the average; the ferromagnet is rebuilt *better* than its own data, the stripe phase is **invisible**
- **C, stripes** ($g \geq 0.8,\ T \leq 1$): the stripe phase and its boundary

Each Regime Sees the Boundary of Its Own Phase



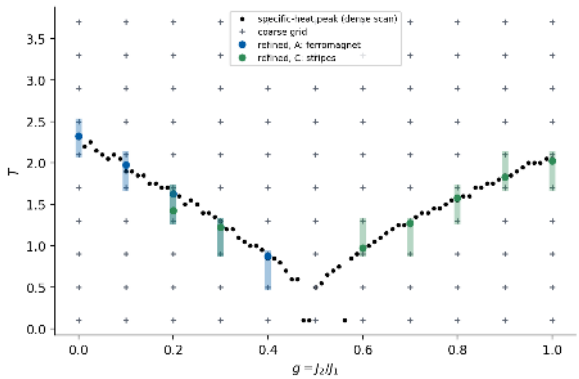
- boundary point: for every g, the temperature where the mean error changes fastest
- A and B: the ferromagnetic line, $0 \leq g \leq 0.49$; C: the stripe line, $0.51 \leq g \leq 1$
- median distance from the specific-heat peak: 0.025, half the temperature step of the grid
- one training region answers one question, *is this like my data?*; it cannot tell two different anomalies apart

Combining the Regimes into a Phase Map



- anomaly score of regime k: $a_k = |\bar{\varepsilon}_k - \mu_k|/\sigma_k$, the distance of the error from its level on the training region; a point goes to the regime with the smallest score, *unassigned* if all are above 3
- the three colours reproduce the three phases
- **unassigned** (21% of the grid): mostly the ordered side of each transition, where fluctuations make snapshots atypical for a model trained deep in the phase
- around $g = 1/2$ at low T: *paramagnet*, correct, since nothing orders there in our temperature range

A Coarse Scan Tells Where to Measure



- coarse grid: $\Delta g = 0.1$, $\Delta T = 0.4$, 10 snapshots per point; regimes A and C trained on the coarse data only
- bars: temperature brackets where the error jumps; dots: boundary after measuring densely *inside the brackets only*
- the dots land on the specific-heat ridge of the dense scan
- cost: $1100 + 2160 = 3260$ snapshots, **0.5%** of the 639 900 used for the dense specific-heat map

Results and Requirements of the Methods

method	input	result vs. the known physics
$\langle \lvert m \rvert \rangle$ scan	the ferromagnetic order parameter	misses the stripe phase
specific heat	many samples per point	both boundaries; expensive
PCA	all snapshots	uniform and stripe patterns: both order parameters
AE, one regime	snapshots of one corner	the boundary of that phase only
AE, three regimes	three corners	three phases; a coarse scan at 0.5% of the cost

The method finds where something changes; which order sets in there is still the physicist's question.

Summary

- **unsupervised learning** finds structure in data without labels: a few coordinates, groups, unusual samples (PCA, t-SNE, k-means, DBSCAN, autoencoders)
- **anomaly detection** with an autoencoder trained on one region locates phase boundaries without labels and without an order parameter
- **2D Ising:** PCA finds the magnetisation; the autoencoder signal converges to T_c with L when trained on the ordered phase; the training region decides the signal
- J_1–J_2**:** the magnetisation misses the stripe phase; PCA finds both order parameters; autoencoders trained on three corners recover the phase diagram, a coarse scan at 0.5% of the cost of a dense one
- the method finds where something changes; which order sets in there is the physicist's question

Next Lecture: Learning a Probability Distribution from Data

The data $\mathbf{x}_1, \ldots, \mathbf{x}_n$ are samples from an unknown distribution $p(\mathbf{x})$. Can we learn p itself, and draw *new* samples from it?

- maximum likelihood, and latent-variable models
- variational autoencoders: new digits from random latent vectors
- conditional generation: a chosen digit, or a spin configuration at a chosen temperature

For the Ising model we know the true distribution, $p(\mathbf{s}) = e^{-H(\mathbf{s})/T}/Z$: a test of what a network has learned.

Tutorial: Hands-on

Hands-on

- run PCA, t-SNE and k-means on MNIST; change the perplexity and the number of clusters k, and see what changes
- train the Ising autoencoder on *disordered* configurations ($T \geq 3$) instead: does the reconstruction error still point to T_c?
- train an autoencoder on all digits except one: is the reconstruction error of the missing digit larger?

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar

